Concept Learning

- concept c: bool-valued func over a set of input instances: +ve / -ve
- ↳ form of supervised learning: correct ans given, infer an unknown bool-valued func from train. eg
- motivation for CL:
  - ↳ simple (noise-free data assump.), easier to fully unds. & exp
  - ↳ white-box model: pred. is interpretable & explainable

- $y = \hat{f}(x) \rightarrow \langle y, c(x) \rangle$ → learn this
- Represent $\hat{f}$ hypothesis ↦ $h \in H$
- one way of rep: h ↔ conjunction of constraints on input attributes ↳ specific value, ?, or ∅
- calc input instance space: $\prod_i^n p_i$, for n attributes, p_i = # possible values of i-th attribute can take
- calc hyp. space:
  - ↳ syntactically distinct: $\prod_i^n (p_i+2)$
  - ↳ semantically distinct: $1 + \prod_i^n (p_i+1)$
  - ↳ ∵ ∀h containing ≥1 ∅ represents an empty set of input instances, hence classifying every instance as -ve

Definitions for concept learning

- hyp space H: $h \in H$ $(h: X \rightarrow \{0,1\})$
- (satisfaction) input instance x ∼ (all constraints of) $h \in H$ iff $h(x) = 1$ ↳ (+ve)
- (satisfaction, -ve) input instance x does not ∼ $h \in H$ iff $h(x) = 0$
- target concept/func: $c: X \rightarrow \{0,1\}$
- Noise-free training eg D: +ve & -ve eg of target func $[D = \{\langle x_k, c(x_k)\rangle\}_{k=1\ldots n}]$
- (consistency) h is ∼ with D iff $h(x) = c(x)$ $\forall \langle x, c(x) \rangle \in D$

Definitions to exploit structure

↳ possibly ↑ → ∞ hyp space & search prob

- (general) h_j more general than or eq to h_k $(h_j \geq_g h_k)$ iff ∀x satisfies h_k also satisfies h_j:
  $\forall x \in X \; (h_k(x) = 1) \rightarrow (h_j(x) = 1)$
  - ↳ partial order, not total order
  - ↳ ∃ h_i & h_j: reflexive $h_i \geq_g h_i$, antisymmetric, transitive
- h_j more general than h_k $(h_j >_g h_k)$ iff $h_j \geq_g h_k$ & $h_k \not\geq_g h_j$
- h_j more specific than (or eq to) h_k iff h_k is more general than (or eq to) h_j

Find-S Algo

1. Initialise h ↦ most specific $h \in H$
2. ∀ +ve x:
   - ↳ ∀ attribute $a_i \in h$:
     - ↳ if x satisfies a_i in h, continue
     - ↳ else replace a_i in h by next more general constraint satisfied by x (minimally gen.)
3. output hyp h

[Proposition 1]: h is consistent w D iff ∀ +ve x satisfies h & ∀ -ve train. x does not satisfy x

[Limitations]: ① determining convergence → c(x) ② not robust to errors / noise ③ picks a maximally specific h & have several ↳ bias to specific hyp c/w initialise most specific & minimally gen.)
↳ ④ failure in complex hyp. space (might be several (none at all) but only choosing one in this algo (backtrack?)

List-then-eliminate

1. VS ← list containing ∀ $h \in H$
2. ∀ train eg $\langle x, c(x) \rangle$:
   ↳ remove from VS any h s.t. $h(x) \neq c(x)$
3. output {} of $h \in VS$

[Limitation]: ex. to exhaustively enum all hyp ∈ H (even finite)

[VS Defined]: $VS_{H,D}$ wrt hyp space H & train eg D is the subset of $h \in H$ consistent with D:
$VS_{H,D} = \{h \in H \mid h \text{ cons. w } D\}$

- if $c \in H$ → large enuf D can ↓ $VS_{H,D}$ to {c}
- if D is insufficient → $VS_{H,D}$ represents uncertainty of what c(x) is
- $VS_{H,D}$ contains ALL h, incl. of maximally specific h

Compact Rep of VS: (Definitions)

- General boundary G of $VS_{H,D}$: set of maximally general members of H cons. w D:
  $G = \{g \in H \mid g \text{ cons. w } D \wedge (\neg \exists g' \in H \; g' >_g g \wedge g' \text{ cons. w } D)\}$
- Specific boundary S: maximally specific $h \in H$ cons:
  $S = \{s \in H \mid s \text{ cons. w } D \wedge (\neg \exists s' \in H \; s >_g s' \wedge s' \text{ cons. w } D)\}$

[Theorem: VSRT]: $VS_{H,D} = \{h \in H \mid \exists s \in S \; \exists g \in G \;\; g \geq_g h \geq_g s\}$

[Proof]: (⇐) RHS
1. choose arbitrary $g \in G$, $s \in S$, $h \in H$ s.t. $g \geq_g h \geq_g s$
2. ∀ +ve train. x satisfies S, by def of S & Prop. 1
3. since $h \geq_g s$, +ve x satisfies h
4. ∀ -ve train eg does not satisfy g, by def of G & prop. 1
5. since $g \geq_g h$, ∀ -ve x does not satisfy h
6. h is cons. w D, by Prop 1. & step 3 & 5
7. $h \in VS_{H,D}$

Candidate elimination Algo

1. G ← max general $h \in H$ 2. S ← max. specific $h \in H$ (lead to)
3. ∀ train eg d:
   - if d is +ve eg:
     - ↳ remove from G h incon. w d
     - ↳ ∀ s ∈ S not cons. w d
       - ↳ Remove s from S
       - ↳ add to S all minimal generalisations h of s s.t. h is cons. w d && ∃ g ∈ G g ≥g h
       - ↳ Remove from S h > g s, s ∈ S, h was ∈ S
   - if d is -ve eg:
     - ↳ remove from S h incon. w d
     - ↳ ∀ g ∈ G not cons. w d
       - ↳ Remove g from G
       - ↳ add to G all minimal specialisations h of g s.t. h is cons. w d && ∃ s ∈ S ≤g h
       - ↳ Remove from G any hyp that is more specific than another hyp ∈ G

[Proof of correctness]: [Tut 1] each minimal gen of h of s is consistent w ∀ +ve & -ve seen thus far (not incl. d)
↳

Limitations of CEA

- S & G reduced to ∅ w sufficiently large data if there is error / noise in D
- Insufficiently exp hyp rep → biased hyp space → c ∉ H ↳ S & G also ↓ ∅ w ↑ data

[Details of CEA] interactively ask human expert to label specific ↑ data pt

- what input instance shld active learner select next to converge fast? → max uncertainty
  - ↳ select x that satisfies exactly half of H in VS
  - ↳ |VS| ↓ by half → ≥ ⌈log₂|VS_{H,D}|⌉ to set c(x)
  - ↳ (similar, minimise sizes of G & S intermediate): select all the d first → ↑ S "out" to ↓ size of VS
  - ↳ or reduce size G 'inwards'
- classify new unobserved input instance
  - ↳ majority vote is the most probable classification, assume all $h \in H$ are equally probable a priori: $P(h) = 1/|H|$

[Unbiased Learner] H that can exp every teachable concept $(2^{|X|})$, no prior assump. made
↳ need train. eg ∀ input instance in X to converge to c(x)
↳ cannot classify new unobserved input instances (generalise)

[Inductive Bias] → resolve limitation of unbiased learner

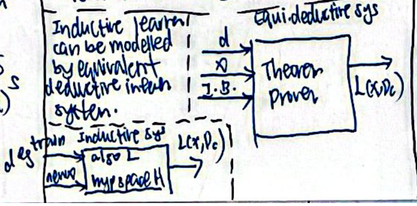
↳ Inductive learning assumpt: any hyp found to approx. the c(x) well over a suffi. large set of train. eg ↦ also approx. c(x) well over other unobserved eg

formally
↳ concept learning algo L, input X, c, noise-free D_c
- let: $L(x, D_c)$:= classification of x by L after learning D_c

||Definition:|| inductive bias of L: any minimal set of assertions B s.t. for any c & D_c:
$\forall x \in X \; (B \wedge D_c \wedge x) \vdash (c(x) = L(x, D_c))$

- Inductive bias of CEA: CEA outputs a classification $L(x, D_c)$ of x if this vote among hyp in VS_{H,D_c} is unanimously +ve / -ve & does not output a classification otherwise | $c \in H$
  ↳ ||Proof||: 1. $c \in H \rightarrow c \in VS_{H,D_c}$ since c cons. w D_c, by def. of VS
  2. L outputs $L(x, D_c) \rightarrow h(x) = L(x, D_c)$ $\forall h \in VS_{H,D_c}$ due to above assumpt, incl. $c \in VS_{H,D_c}$ by step 1
  3. $c(x) = L(x, D_c)$
  → stronger bias, greater gen. power (over unobserved)
- Rote learner: no inductive bias
- Find-S (more strong bias): $c \in H$ & all x are -ve unless the opp. is entailed by its other knowledge

Inductive learner can be modelled by equivalent deductive inferen system.



Bayesians

$P(h|D) = \frac{P(D|h)\,P(h)}{P(D)}$

- P(h): prior belief of h (prior prob.)
- P(D|h): likelihood of D given h (exp. of h)
- $P(D) = \sum_{h \in H} P(D|h)P(h)$: marginal likelihood / evidence of D
  ↳ D↑, more h can exp h, P(D|h) ↓
- P(h|D): posterior prob / belief of h after seeing D

Limitations:
- requires specifying prob & underlying dist
  - ↳ (i) some P(h) = 0 - wrong posterior?
  - ↳ (ii) likelihood func must be chosen correctly
  - ↳ (iii) need data to justify prior
- often prohibitively ex to compute evi

[Choosing hyp] max. a posteriori:

$h_{MAP} = \underset{h \in H}{\arg\max}\, P(h|D)$
$= \underset{h \in H}{\arg\max} \frac{P(D|h)P(h)}{P(D)}$ → same
$= \underset{h \in H}{\arg\max}\, P(D|h)P(h)$

- max. likelihood: (if $P(h) = P(h')$ $\forall h, h' \in H$)
  $h_{ML} = \underset{h \in H}{\arg\max}\, P(D|h)$
  ↳ consistent hyp can be ML but not MAP / also can output cons. hyp not MAP, depends on algo.

P(D) = P(+ve/-ve), P(h) = P(C)

Basic Prob Formula

- chain rule: $P(A_1, \ldots, A_n) = \prod_{i=1}^n P(A_i | A_1, \ldots, A_{i-1})$
- incl-excl: $P(\bigcup_{i=1}^n A_i) = \sum_{1 \leq i \leq n} P(A_i) - \sum_{1 \leq i < j \leq n} P(A_i, A_j) + \sum_{1 \leq i < j < k \leq n} P(A_i, A_j, A_k) \ldots + (-1)^{n-1} P(A_1, \ldots, A_n)$
- Marginalisation: $A_1, \ldots, A_n$ mutually exclusive s.t. $\sum_{i=1}^n P(A_i) = 1$
  → $P(B) = \sum_{i=1}^n P(B|A_i)P(A_i)$ (law of total prob)

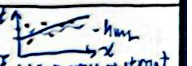
Bayesians X concept learning

- (simplest) assume x_d, $d \in D$ are fixed
- $P(D|h) = \begin{cases} 1, & \text{if h is consistent w } D \\ 0 & \text{otherwise} \end{cases} = \delta$
- $P(h) = \frac{1}{|H|}$ $\forall h \in H$
  ↳ $P(h|D) = \begin{cases} \frac{1}{|VS_{H,D}|}, & \text{if h cons. w } D, \\ 0 & \text{otherwise} \end{cases}$
  [every cons. hyp is a MAP]

MAP Learning Inductive System

P(h) uniform, P(D|h) = δ

Learning a cont.-valued function



- real-valued target f & $D = \{(x_d, t_d)\}$, t_d is noisy output for of
  $t_d = f(x_d) + \epsilon_d$
  ↳ $\epsilon_d \sim N(0, \sigma^2)$ → noise var drawn ⊥ $\forall x_d$

Gaussian: $f(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$

$\Rightarrow h_{ML} = \underset{h \in H}{\arg\min} \frac{1}{2} \sum_{d \in D} (t_d - h(x_d))^2$ — show they are equivalent
even ↳ $h_{ML} = \underset{h \in H}{\arg\max}\, P(D|h)$ → $P(t_d | f(x_d)) \sim N(f(x_d), \sigma^2)$ ↓ $h(x_d)$ → $N(t_d | h(x_d), \sigma^2)$
$= \underset{h \in H}{\arg\max} \prod_{d \in D} P(t_d | h, x_d)$
$= \underset{h \in H}{am} \prod_{d \in D} \frac{1}{\sqrt{2\pi}\sigma} \exp\left(-\frac{(t_d - h(x_d))^2}{2\sigma^2}\right)$ — take log (sum of log, monotonic)
$= \underset{h \in H}{am} \sum_{d \in D} \ln \frac{1}{\sigma\sqrt{2\pi}} - \frac{(t_d - h(x_d))^2}{2\sigma^2}$ ($\ln e^a = a$) — remove constant term
$= \underset{h \in H}{am} \sum_{d \in D} -\frac{(t_d - h(x_d))^2}{2\sigma^2}$ — remove constant, keep ½ for $\frac{\partial f}{\partial x}$
$= \underset{h \in H}{am} \frac{1}{2} \sum_{d \in D} -(t_d - h(x_d))^2$ — min neg log-likelihood [argmax = argmin]
$= \underset{h \in H}{\arg\min} \frac{1}{2} \sum_{d \in D} (t_d - h(x_d))^2$ → convex loss fn! grad descent

Learn Predict Prob

c(x), $c: X \rightarrow \{0,1\}$ (boolean)
↳ neural network output $P(c(x) = 1)$ via h_{ML}:
$h_{ML} = \underset{h \in H}{\arg\max} \sum_{d \in D} t_d \ln h(x_d) + (1 - t_d) \ln(1 - h(x_d))$
↳ prob!, not 0, 1

justify (derive): where assume x_d not known

pred. prob
- $P(D|h) = \prod_{d \in D} P(x_d, t_d | h) = \prod_{d \in D} P(t_d | h, x_d) P(x_d)$, $x_d \perp h$
- $P(t_d | h, x_d) = \begin{cases} h(x_d) & \text{if } t_d = 1 \\ 1 - h(x_d) & \text{if } t_d = 0 \end{cases}$
  $= h(x_d)^{t_d} (1 - h(x_d))^{1 - t_d}$ — substitute

- || $h_{ML} = \underset{h \in H}{\arg\max}\, P(D|h)$
  $= \underset{h \in H}{\arg\max} \prod_{d \in D} h(x_d)^{t_d} (1 - h(x_d))^{1 - t_d} P(x_d)$ — remove constant term
  $= \underset{h \in H}{am} \prod_{d \in D} h(x_d)^{t_d} (1 - h(x_d))^{1 - t_d}$ — take log
  $= \underset{h \in H}{am} \sum_{d \in D} t_d \ln h(x_d) + (1 - t_d) \ln(1 - h(x_d))$

↓ leads to

[Gradient Ascent to max likelihood in a sigmoid unit] (→ activation fn)

$U_D(h) = \sum_{d \in D} t_d \ln h(x_d) + (1 - t_d) \ln(1 - h(x_d))$

$\frac{\partial U_D}{\partial w_i} = \sum_{d \in D} \frac{\partial U_D}{\partial h(x_d)} \cdot \frac{\partial h(x_d)}{\partial w_i}$ — differentiate sigmoid fn
$= \sum_{d \in D} \left[\frac{(t_d - h(x_d))}{h(x_d)(1 - h(x_d))}\right] \left[h(x_d)(1 - h(x_d)) x_{id}\right]$
$= \sum_{d \in D} (t_d - h(x_d)) x_{id}$ — Gradient Ascent: max (reward)

$w_i \leftarrow w_i + \Delta w_i$, $\Delta w_i = \eta \frac{\partial U_D}{\partial w_i}$ — Gradient Descent: min (loss fn)

Minimum Description Length

Occam's razor: prefer shortest hyp that fits data

$h_{MAP} = \arg\max_{h \in H} P(D|h)P(h)$

log trick: $= \arg\max_{h \in H} \log_2 P(D|h) + \log_2 P(h)$

min neg log likelihood: $= \arg\min_{h \in H} -\log_2 P(D|h) - \log_2 P(h)$

- $-\log_2 P(h)$: description len of h under optimal code
- $-\log_2 P(D|h)$: descr. len of D given h

MDL: $h_{MDL} = \arg\min_{h \in H} L_{C_1}(h) + L_{C_2}(D|h)$

$L_C(x)$: desc. len of x under encoding C

- MDL example: decision trees = H
  - $L_{C_1}(h)$:= # bits to describe tree h
  - $L_{C_2}(D|h)$:= # bits for D given h
    - 0 if classified correctly by h, misclassification: $\frac{\log_2 |D|}{\log_2 |T|}$ bits at most (class labels)

implication: min $L_C(h)$ & $L_C(D|h)$ (min misclassifications)

h_{MDL} trades off h size for training errors > mitigate overfitting (noisy D)

Most Probable classification x

h_{MAP} - most prob. hyp but not most prob. classification!

consider multiple hyp/experts

Bayes-optimal classifier

$\arg\max_{t \in T} P(t|D) = \arg\max_{t \in T} \sum_{h \in H} P(t|h)P(h|D)$

Example: $T = \{+, -\}$

$P(h_1|D) = .4 \quad P(-|h_1) = 0 \quad P(+|h_1) = 1$

$P(h_2|D) = .3 \quad P(-|h_2) = 1 \quad P(+|h_2) = 0$

$P(h_3|D) = .3 \quad P(-|h_3) = 1 \quad P(+|h_3) = 0$

$\sum_{h \in H} P(+|h)P(h|D) = .4 \quad \sum_{h \in H} P(-|h)P(h|D) = .6$

$\arg\max_{t \in \{+,-\}} \sum_{h \in H} P(t|h)P(h|D) = -ve$

proof of performance:

- BOC is best perf (tho computationally costly w/ ↑H)
- max(min) prob of correctly (incorrectly) classifying a new input instance
- [Proof] (expected 0-1 loss) (loss fn) $L_D(t^*) = \sum_{t \in T} (1 - \delta_{tt^*}) P(t|D)$

$\delta_{tt^*} = \begin{cases} 1, & t = t^* \\ 0, & \text{otherwise} \end{cases}$

$= \sum_{t \in T} [P(t|D) - \delta_{tt^*} P(t|D)]$

only ≠0 correct classification survives: $= 1 - P(t = t^*|D)$

(error fn) $= P(t \neq t^*|D)$

≡ to BOC: argmin$_{t \in T}$ min prob. of misclassification

$t_{BOC} = \arg\max_{t \in T} P(t = t^*|D)$

$= \arg\min_{t \in T} L_D(t^*)$

Gibbs algo

- solve computationally costly BOC
  - sample a h from posterior belief $P(h|D)$
  - use h to classify new x
  - surprising result: if c are sampled from some prior over H, expected misclassification error of Gibbs classifier: $P(t \neq t^*|D)_G \geq 2 \cdot P$ of BOC

Naive Bayes Classifier — diagnosis

- Limitation: moderate/↑ train data + assumption: input attr. are conditionally ⊥ given classification
- $c: X \to T$, each $x \in X$ is $x = (x_1, \ldots, x_n)^T$
- most probable classification:

$t_{MAP} = \arg\max_{t \in T} P(t|x_1, \ldots, x_n)$

$= \arg\max_{t \in T} \frac{P(x_1, \ldots, x_n|t)P(t)}{P(x_1, \ldots, x_n)}$

$= \arg\max_{t \in T} P(x_1, \ldots, x_n|t)P(t)$

→ NB assumption: $P(x_1, \ldots, x_n|t) = \prod_{i=1}^{n} P(x_i|t)$

$\therefore t_{NB} = \arg\max_{t \in T} P(t) \prod_{i=1}^{n} P(x_i|t)$

- space efficiency: if $\forall$ attri ∈ 2 values
  $P(x_i|t) \ldots P(x_j|t=0)$, 0/1 0/1 … x_n: n, 2n (to given t=1)

VS No NB assumption: $P(x_1, \ldots, x_n|t)$

→ $2^n - 1 \times 2$ (2t values) −1 for $P(x_1 = 0, \ldots x_n = 0|t)$ case

- Data efficient:
  NB: $P(x_i|t)$ for $i = 1, \ldots, n$ − $2n \times 2$
  Non-NB: $P(x_1, \ldots, x_n|t)$ − $2^n \times 2$

NB Algo

NB-Learn(D):

- For each value of target output t
  - $\hat{P}(t) \leftarrow$ estimate $P(t)$ using D
  - For each value of attr. x_i
    - $\hat{P}(x_i|t) \leftarrow$ est. $P(x_i|t)$ w/ D

(freq. count)

classify (x):

$t_{NB} = \arg\max_{t \in T} \hat{P}(t) \prod_{i=1}^{n} \hat{P}(x_i|t)$

Properties of NB

1. cond. ⊥ assump often violated but NB still works well: $\hat{P}(t|x)$ need not be correct
   → $\arg\max_{t \in T} \hat{P}(t) \prod_{i=1}^{n} \hat{P}(x_i|t) \approx \arg\max_{t \in T} P(t)P(x_1, \ldots, x_n|t)$
2. If $\hat{P}(x_i|t) = 0$ (sparse data) → $\hat{P}(t) \prod_{i=1}^{n} \hat{P}(x_i|t) = 0$ (dominate) + biased estimate of true data
   Solution: est. $\hat{P}(x_i|t) \leftarrow \frac{|D_{tx_i}| + mp}{|D_t| + m}$
   - p: prior (es $\frac{1}{|H|}$) for $\hat{P}(x_i|t)$
   - m: weight given to p (virtual eg)

EM (2 steps)

- When to use: (semi-supervised learning (both labelled & unlabelled to refine params est))
  - partially observable data
  - unsupervised clustering (target unobservable)
  - supervised learning (some input attri. unob.)
- Generate data from mixture of M Gaussians
  - each x_d is generated via ① select one of M, with uniform prob ② sample an x_d from that Gaussian
- Known: $x_d (\in X) \sim N(\mu, \sigma^2)$ (σ² known)
  unknown: gaussian means $\langle \mu_1, \ldots, \mu_M \rangle$
  ② which x_d is generated by which Gaussian (membership)
  → Figuring out ② could be helpful
- Set up ∴:
  - determine ML est. of $\langle \mu_1, \ldots, \mu_M \rangle$
  - consider full descrip. of each instance as $d = \langle x_d, z_{d1}, z_{d2}, \ldots z_{dM} \rangle$
    where: ① z_{dm} is unobservable & is of value 1 if mth Gaussian is selected to generate x_d, 0 otherwise (indicator var) ② x_d is observable

EM algo

: pick rand. initial $h = \langle \mu_1, \mu_2 \ldots \rangle$ then iter.

- E step: calc $E[z_{dm}]$ of each hidden/latent var z_{dm}, assuming current h holds
  → $E[z_{dm}] = \frac{P(x_d|\mu_m)}{\sum_{l=1}^{M} P(x_d|\mu_l)} = \frac{\exp(-\frac{1}{2\sigma^2}(x_d - \mu_m)^2)}{\sum_{l=1}^{M} \exp(-\frac{1}{2\sigma^2}(x_d - \mu_l)^2)}$
  → prob that mth gaussian is selected given x_d is generated
- M step: calc $h' = \langle \mu'_1, \ldots, \mu'_M \rangle$, assume $z_{dm} = E[z_{dm}]$ in E step. Replace $h \leftarrow h' = \langle \mu'_1, \ldots \rangle$
  → $\mu'_m \leftarrow \frac{\sum_{d \in D} E[z_{dm}] x_d}{\sum_{d \in D} E[z_{dm}]}$ (contrib. of each input x to μ_m)

- Derivation for $E[z_{dm}]$
  → $E[z_{dm}|h, x_d] = 1 \times P(z_{dm} = 1|h, x_d) + 0 \times P(z_{dm} = 0 \ldots)$

Bayes Rule: $= \frac{P(x_d|h, z_{dm}=1) P(z_{dm}=1|h)}{\sum_{l=1}^{M} P(x_d|h, z_{dl}=1) P(z_{dl}=1|h)}$

uniform priors ($P(z_{dm}=1|h)$): $= \frac{P(x_d|h, z_{dm}=1)}{\sum_{l=1}^{M} P(x_d|h, z_{dl}=1)}$ (choose mth μ)

$= \frac{P(x_d|\mu_m)}{\sum_{l=1}^{M} P(x_d|\mu_l)}$

- [concrete application of EM in est. M means]
- Properties: → converge to local ML hyp h' (max. likelihood) & provides est. of z_{dm}
  → local max in $E[\ln p(D|h')]$
  - D is complete (obs. + unobs.) data
  - Expectation w.r.t. unobs. var in D

Applying EM to est. M means: Derivation

- (derivation of max.) 1. $p(d|h') = p(\underbrace{x_d, z_{d1}, \ldots, z_{dM}}_{D}|h')$

$= \frac{1}{\sqrt{2\pi}\sigma} \exp(-\frac{1}{2\sigma^2} \sum_{m=1}^{M} z_{dm}(x_d - \mu'_m)^2)$

→ only one has value, by right summation can disappear but keep for differentiation

→ one hot product trick (z_{dm}, one-hot encoded indicator var):

$p(x_d, z_{d1}, \ldots, z_{dM}|h) \propto \prod_{m=1}^{M} [N(x_d|\mu_m, \sigma^2)]^{z_{dm}}$

- $z_{dm} = 0$ → become 1 & does not affect product
- $z_{dm} = 1$ → active prob term

$p(d|h') = \prod_{m=1}^{M} [\frac{1}{\sqrt{2\pi}\sigma} \exp(-\frac{1}{2\sigma^2}(x_d - \mu'_m))]^{z_{dm}}$

(by dist. power) $= \prod_{m=1}^{M} (\frac{1}{\sqrt{2\pi}\sigma})^{z_{dm}} \cdot \prod_{m=1}^{M} \exp(-\frac{1}{2\sigma^2}(x_d - \mu'_m)^2)^{z_{dm}}$

($z_{dm} = 0$, $= 1$) & $(e^A)^B = e^{AB}$: $= \frac{1}{\sqrt{2\pi}\sigma} \prod_{m=1}^{M} \exp(-\frac{1}{2\sigma^2} z_{dm}(x_d - \mu'_m)^2)$

($\prod e^{A_i} = e^{\sum A_i}$): $= \frac{1}{\sqrt{2\pi}\sigma} (\sum_{m=1}^{M} -\frac{1}{2\sigma^2} z_{dm}(x_d - \mu'_m)^2)$ ($\frac{1}{2\sigma^2}$ constant)

$\prod a^{A_i} = a^{\sum A_i}$

$\ln p(D|h') = \ln \prod_{d \in D} p(d|h') = \sum_{d \in D} \ln p(d|h')$

$= \sum_{d \in D} (\ln \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{2\sigma^2} \sum_{m=1}^{M} z_{dm}(x_d - \mu'_m)^2)$

$Q(h'|h) = E[\ln p(D|h')]$

$= E[\sum_{d \in D} (\ln \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{2\sigma^2} \sum_{m=1}^{M} z_{dm}(x_d - \mu'_m)^2)]$

(linearity of E) $= \sum_{d \in D} (\ln \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{2\sigma^2} \sum_{m=1}^{M} E[z_{dm}](x_d - \mu'_m)^2)$

- Max. step: $\arg\max_{h'} Q(h'|h)$

$= \arg\max_{h'} \sum_{d \in D} (\ln \frac{1}{\sqrt{2\pi}\sigma} - \frac{1}{2\sigma^2} \sum_{m=1}^{M} E[z_{dm}](x_d - \mu'_m)^2)$

$= \arg\max_{h'} \sum_{d \in D} - \sum_{m=1}^{M} E[z_{dm}](x_d - \mu'_m)^2$ ← remove constant term

$= \arg\min_{h'} \sum_{d \in D} \sum_{m=1}^{M} E[z_{dm}](x_d - \mu'_m)^2$ ← max neg func

→ Derivation

$\mu'_m \leftarrow \frac{\sum_{d \in D} E[z_{dm}] x_d}{\sum_{d \in D} E[z_{dm}]}$

$\frac{\partial}{\partial \mu'_m} \sum_{d \in D} \sum_{m=1}^{M} E[z_{dm}](x_d - \mu'_m)^2$

$= \sum_{d \in D} E[z_{dm}] \frac{\partial}{\partial \mu'_m}(x_d - \mu'_m)^2$ ← $E[z_{dm}] = 1$ iff m is selected

$= -2 \sum_{d \in D} E[z_{dm}](x_d - \mu'_m)$ ← diff. rule

$= -2(\sum_{d \in D} E[z_{dm}] x_d - \mu'_m \sum_{d \in D} E[z_{dm}])$ ← distr. power

$= 0$ → local max $\frac{d}{d\mu} = 0$

$\mu'_m = \frac{\sum_{d \in D} E[z_{dm}] x_d}{\sum_{d \in D} E[z_{dm}]}$ ← rearrange

General EM Algo | $Q(h'|h) := E[\ln p(D|h') | h, \{x_d\}_{d \in D}]$

- Pick rand. initial h, then iterate:
- E-step: calc. $Q(h'|h)$ using h & $\{x_d\}_{d \in D}$ to est. latent var $\{z_d\}_{d \in D}$:
  $Q(h'|h) \leftarrow E[\ln p(D|h') | h, \{x_d\}_{d \in D}]$
  (why calls it max step)
- M step: replace h w/ h' that max Q:
  $h \leftarrow \arg\max_{h'} Q(h'|h)$

Given: observed data $\{x_d\}_{d \in D}$ ② unobs. data $\{z_d\}_{d \in D}$ where $z_d = \langle z_{d1}, \ldots, z_{dM} \rangle$, ③ param. prob. dist. $P(D|h)$ where

- $D = \{d\}$ – complete data $\langle x_d, z_d \rangle$
- h comprises params

[concrete application of General EM to est. M mean]

- Given: x ~ X generated by mixture of M gaussians w/ known σ² [$\{x_d\}_{d \in D}$]
  - unknown $\langle \mu_1, \ldots, \mu_M \rangle$ [$\{z_d\}_{d \in D}$]
  - unknown membership [$P(D|h)$]
- Determine: max likelihood (ML) est of μ

$P(h|D_1, D_2) = \frac{P(D_1, D_2|h)P(h)}{P(D_1, D_2)} = \frac{P(D_2|h)P(h|D_1)}{\sum_{h \in H} P(D_2|h)P(h|D_1)}$

conditional ⊥ of D_1 & D_2 given h

- why $E[\ln p(D|h') | h, \{x_d\}_{d \in D}] > \ln p(x_D|h')$ (max log-likelihood of obs. data & marginalize out the unobs. var Z_D)

∴ proposition: log likelihood of obs. data monotonically ↑ w/ an ↑ # of EM iterations

① using $p(x_D|h')$ is complicated: $\ln(\sum_{Z_D} p(x_D, Z_D|h'))$

$\ln p(x_D|h') = \ln P(x_D, Z_D|h') - \ln P(Z_D|h', x_D)$

($\because p(x_D|h') = p(x_D, Z_D|h') / P(Z_D|h', x_D)$)

$\sum_{Z_D} P(Z_D|h, x_D) \ln p(x_D|h') = \sim$ ($\sum_{Z_D} P(Z_D|h, x_D) = 1$)

: $\ln p(x_D|h') = \sum_{Z_D} P(Z_D|h, x_D) \ln p(x_D, Z_D|h') = Q(h'|h)$

$- \sum_{Z_D} P(Z_D|h, x_D) \ln p(Z_D|h', x_D) := R(h'|h)$

$\ln p(x_D|h) = Q(h|h) + R(h|h)$ → subs. h to (1)

$\ln p(x_D|h') - \ln p(x_D|h) = Q(h'|h) - Q(h|h) + R(h'|h) - R(h|h)$ (1)−(2)

log of curr. iteration is at least that of prev iteration ← $\ln p(x_D|h') - \ln p(x_D|h) \geq Q(h'|h) - Q(h|h)$ by Gibbs' ineq: $R(h'|h) \geq R(h|h)$

COLT

COLT → relax from concept learning: Prob of successful learning is a few factors

- determining $h \in H$ consistent w D vs [training] $h \in H$ consistent w $\{\langle x, c(x)\rangle\}_{x \in X}$ [test also]
- [Sample complexity]: # data required to achieve a specific level of performance, confidence in learning c, generalize)

[Setting] 1. exact case, VS = {c}:
- active learner repeatedly selects x to query a teacher for c(x) — how many queries needed to guarantee VS = {c}
- 2. teacher chooses & selects $d = \langle x, c(x)\rangle$ — how many instances needed to guarantee VS = {c}?

3. approx. case: random process repeatedly generates x to query teacher for c(x) — how many queries needed to guarantee $h \approx c$

[SETTING 1] opt query strategy:
- ↳ select x that satisfies exactly half of hyp in VS (if possible)
- ↳ $|VS| \downarrow \frac{1}{2}$ w d, require $\geq \lceil \log_2 |VS_{H,D}| \rceil$ examples to find C (converges)
- → CEA of concept learning for eg

[SETTING 2] opt teaching strategy:
- depends on H used by learner
- ↳ eg: H = conj up to n Bool literals + negations & up to k specific literals in n-k don't cares
- [at most k+2 d suffices (proven by induction)

[SETTING 3] X → input instances, H → hyp, C is possible target func
- Learner observe set D of noise-free training eg $\langle x, c(x)\rangle$ of some $c \in C$, x is randomly sampled from Q
- learner output h to approx. C; h is eval by performance on new input x sampled from Q ("testing data")
- True error: $error_Q(h)$ of h w.r.t. C & Q is $P_{x \sim Q}(h(x) \neq c(x))$ [measured wrt to entire Q]
- Training error: $error_D(h) = \frac{1}{|D|} \sum_{x \in D} (1 - \delta_{h(x), c(x)})$ of h w.r.t. C where $\delta_{h(x)c(x)} = \begin{cases} 1, & \text{if } h(x) = c(x) \\ 0, & \text{otherwise} \end{cases}$

[Bounding $error_Q(h)$]
- (Definition). $VS_{H,D}$ is ε-exhausted wrt C & Q iff $\forall h \in VS_{H,D}$, $error_Q(h) < \varepsilon$
- (Theorem 1). If H is finite & D is set of ⊥ rand. examples ($|D| \geq 1$) some some C, then for any $0 \leq \varepsilon \leq 1$, $P_Q[VS_{H,D}$ not ε-exhausted$] \leq |H| \exp(-\varepsilon |D|)$

ans: how many d will ε-exhaust $VS_{H,D}$

[Bounding $error_Q(h)$ - cont]
- Theorem 1 limitation - bound is loose due to large (infinite) H
- ↳ implies: pr of outputting consistent h w $error_Q(h) \geq \varepsilon$
- ↳ Goal: $|H| \exp(-\varepsilon |D|) \leq \delta \Rightarrow |D| \geq \frac{1}{\varepsilon}[\ln|H| + \ln(\frac{1}{\delta})]$
- ↳ ε ≤ [corollary 1.] $0 < \varepsilon, \delta \leq 1$: if H is finite & D is a set of independent rand. eg of some c s.t. $|D| \geq \frac{1}{\varepsilon}[\ln|H| + \ln(\frac{1}{\delta})]$ → $P(\forall h \in VS_{H,D}\ error_Q(h) < \varepsilon) \geq 1 - \delta$

[Theorem 1 Proof]
1. $VS_{H,D}$ is not ε-exhausted iff $\exists h \in VS_{H,D} \wedge error_Q(h) \geq \varepsilon$ w.r.t. to C (by def. of ε-exhausted)
2. $error_Q(h) = P_{x \sim Q}(h(x) \neq c(x)) \geq \varepsilon$ (single eg x)
3. $P_{x \sim Q}(h(x) = c(x)) \leq 1 - \varepsilon$ (prob that h w $error_Q(h) \geq \varepsilon$ is consistent w one rand eg is at most $1-\varepsilon$)
4. $P[h$ is $error_Q(h) \geq \varepsilon$ const w $|D|$ independent rand eg$]$: $P(h \in VS_{H,D} \wedge error_Q(h) \geq \varepsilon) \leq (1-\varepsilon)^{|D|}$
5. $P(\exists h \in H\ h \in VS_{H,D} \wedge error_Q(h) \geq \varepsilon) \leq |H|(1-\varepsilon)^{|D|}$, by union bound
6. $P(VS_{H,D}$ not ε-exhausted$) \leq |H|(1-\varepsilon)^{|D|} \leq |H| \exp(-\varepsilon |D|)$ by step 1 & $(1-\varepsilon) \leq \exp(-\varepsilon)$ for any $0 \leq \varepsilon \leq 1$

[PAC Learning] · class C, X input instances (n), learner L w H
- class C is PAC-learnable by L using H iff $\forall c \in C$, distr Q over X, $0 < \varepsilon, \delta < 1$, prob that L outputs $h \in H$ with $error_Q(h) \leq \varepsilon$ is at least $1-\delta$ in time that is polynomial in $1/\varepsilon$, $1/\delta$, n, & size(C)
- ↳ given C is by L, L incurs some min time to process each training eg → L learns from a polynomial # eg
- ↳ to show C is PAC-learnable by L, show that each $c \in C$ can be learned from a poly # training eg using polynomial time per training eg

Theorem 2. C is PAC by FIND-S using H = C (C is ∧ up to n Bool literals & their negations)

Proof: 1. $\forall c \in C$, $P(\forall h \in VS_{H,D}\ error_Q(h) < \varepsilon) \geq 1-\delta$ in # eg that is poly in n, $1/\varepsilon$, $1/\delta$, & ⊥ of size(c) by corollary 1 & example 1 on pg 15
2. $H \subseteq C$, Pr FIND-S outputs $h \in VS_{H,D} \subseteq H$ w $error_Q(h) < \varepsilon$ is at least $1-\delta$ in # eg as in step 1
3. To process each eg, FIND-S incurs time linear in n & ⊥ of $1/\varepsilon$, $1/\delta$, size(c)
4. $H \in C$, Pr FIND-S outputs $h \in VS_{H,D} \subseteq H$ w $error_Q(h) < \varepsilon$ is at least $1-\delta$ in time poly in n, $1/\varepsilon$, $1/\delta$ & ⊥ of size(c) by step 2 & 3

S ∴ C is PAC-learnable by FIND-S, by implication 2

[Agnostic Learning] $c \in H$ is not assumed → agnostic L output h w min train error
- ↳ how # training eg suffice to guarantee $error_Q(h) < error_D(h^*) + \varepsilon$ w prob at least $1-\delta$?
1. $P(error_Q(h) \geq error_D(h) + \varepsilon) \leq \exp(-2|D|\varepsilon^2)$ by Hoeffding's
2. $P(\exists h \in H\ error_Q(h) \geq error_D(h) + \varepsilon) \leq |H| \exp(-2|D|\varepsilon^2)$ by U
3. # |D| to ↓ this prob to at most δ: $|H| \exp(-2|D|\varepsilon^2) \leq \delta$
4. Then, $|D| \geq \frac{1}{2\varepsilon^2}[\ln|H| + \ln(\frac{1}{\delta})]$
- ↳ price paid for relaxing the assumption [↑ term more when ↓]
- ↳ compared to corollary 1

|H| is infinite?! - VC & shattering

[Shattering] S: a dichotomy (Y, S\Y) of set S is a partition of S into 2 disjoint subsets $Y \in 2^S$ & S\Y
- a hyp $h \in H$ is consistent w a dichotomy (Y, S\Y) of a set S of input instances iff $(\forall x \in Y\ h(x) = 1) \wedge (\forall x \in S \backslash Y\ h(x) = 0)$
- A set of input instances $S \subseteq X$ is shattered by H iff ∀ dichotomy of S, $\exists h \in H$ that is consistent with this dichotomy (Y)
- ↳ Implication: ∃ some dichotomy s.t. no $h \in H$ is consistent w AND some dichotomy does not have a consistent $h \in H$ (∃)
- ↳ unbiased hyp space: shatter whole input space (2^X) ↳ but cannot generalize [i.e. find X as big as possible]

- Vapnik-Chervonenkis (VC) dimension
- ↳ VC(H) of H defined over instance space X is the size of largest finite subset of X shattered by H
- --→ if arbitrary large finite set of X can be shattered by H → VC(H) = ∞
- Proposition 1: ∀ finite H, $VC(H) \leq \log_2 |H|$
- ↳ implication: sample complexity can be ↓ if ln|H| term can be replaced by VC(H)
- Theorem 3. Let $0 < \varepsilon, \delta < 1$. If D is a set of ⊥ rand examples of some C s.t. $|D| \geq (\frac{1}{\varepsilon})(8\,VC(H)\log_2(\frac{13}{\varepsilon}) + 4\log_2(\frac{2}{\delta}))$ then: $P(\forall h \in VS_{H,D}\ error_Q(h) < \varepsilon) \geq 1-\delta$

(Proposition 1 Proof): suppose VC(H) = d
2. By def. d instances is largest finite ⊆ X shattered by H (h(x)=0 or h(x)=1)
3. H requires at least 2^d distinct hyp to shatter d instances (may be semantically similar)
4. ⇒ $H \geq 2^d \leftrightarrow \log_2 H \geq d = VC(H)$

[Tricks for finding VC(H)]
1. If $X = \mathbb{R}$, x-y plane, x-y-z plane → consider geometric intuition:
- → what is H (a line, a point, a plane…)
- → VC of linear decision surface in r-dim space is r+1
- → draw out & visualise its between X & H (eg X = x-y plane, H = linear decision, cannot shatter collinear 3 points)

2. Show VC(H) ≤ d can → there exists |VC(H)| > d cannot, $\forall S \subseteq X, |S| > d$, S is not shattered by H

[MDP] - models sequential decision problem (w uncertainties)

[Model Formulation]
MDP ≜ (S, A, T, R) consists of:
- a set S of states
- a set A of actions
- a transition func $T: S \times A \times S \to [0,1]$ s.t. $\forall s \in S\ \forall a \in A\ \sum_{s' \in S} T(s,a,s') = 1$
- a reward func $R: S \to \mathbb{R}$

MDP (cont)

- $T(s,a,s') = P(s'|s,a)$ → prob of going from states to state s' using action a.
- ↳ define T(s, a, s') for all $s, s' \in S$, $a \in A$
- ★ Markov property: next state is cond. ⊥ of the past states & actions given curr state & action: $P(s_{t+1}|s_t, a_t, \ldots, s_0, a_0) = P(s_{t+1}|s_t, a_t)$ $\forall s_{t+1}, s_t, a_t, \ldots, s_0, a_0$
- ↳ have violation (when ∃ unmodeled state var & dynamics of env, inaccurate in transition func)
- ↳ but this assumption still preferred in order to reduce time complexity of algorithms
- $O(|S|^t |A|^t)$ vs $O(|S|^2 |A|)$

[Reward function] · Define R(s) for all $s \in S$
- ↳ Forms: R(s), R(s,a), R(s,a,s')

[Utility of state seq]
- Additive rewards: $U_h([s_0, s_1, \ldots]) = R(s_0) + R(s_1) + \ldots +$ ↳ utility of an infinite state seq is infinite
- Discounted rewards: $U_h([s_0, s_1, \ldots]) = R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \ldots$ where $\gamma \in [0,1]$ is the discount factor
- ↳ if $\gamma < 1$ & $R(s_t) \leq R_{max}$ then: $U_h([s_0, s_1, \ldots]) = \sum_{t=0}^{\infty} \gamma^t R(s_t) \leq \sum_{t=0}^{\infty} \gamma^t R_{max} = R_{max}/(1-\gamma)$
- ↳ infinite state seq w finite utility
- Geometric sum: $S_n = a\frac{1-r^n}{1-r}$, $S_\infty = \frac{a}{1-r}$
- ↳ search prob: find optimal seq
- ↳ a policy: $\pi \triangleq \{\pi(s)\}_{s \in S}$ where $\pi: S \to A$
- ↳ MDP, aim to find opt $\pi^* \triangleq \{\pi^*(s)\}_{s \in S}$ (best action for state s) → max E(U)

[Utility of State] (under a policy π)

$U^\pi(s) \triangleq \mathbb{E}[U_h([s_0, s_1, \ldots]) | \pi, s_0 = s]$ starting in s & following π

$= \mathbb{E}[\sum_{t=0}^{\infty} \gamma^t R(s_t) | \pi, s_0 = s]$

- ↳ solving MDP - choose π that max $U^\pi(s)$ which yields opt π^*: $\pi^* = \arg\max_\pi U^\pi(s)$

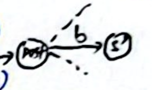
[Bellman Equation]: utility of a state is immediate reward of the state + expected discounted utility of next state, given agent chooses the opt. action

$U(s) \triangleq U^{\pi^*}(s)$: $U(s) = R(s) + \gamma \max_{a \in A(s)} \sum_{s'} T(s,a,s') U(s')$
- $U(s) = \max_{a \in A} R(s,a) + \gamma \sum_{s'} T(s,a,s') U(s')$
- $U(s) = \max_{a \in A} \sum_{s' \in S} T(s,a,s')[R(s,a,s') + \gamma U(s')]$
- → $\pi^*(s) = \arg\max_{a \in A(s)} \sum_{s'} T(s,a,s') U(s')$
- to convert from diff. forms, ensure the opt π remains unchanged under this new form ↳ state splitting trick
- Eg: conversion: $R(s,a,s') \to R(s,a)$ form:
- ↳ new model: $A \cup \{b\}$, $S \cup \{pre(s,a,s')\}_{s \in S, a \in A, s' \in S}$
1. T', R' and γ' → new model
2. $T'(s,a,pre(s,a,s')) = T(s,a,s')$
3. $T'(pre(s,a,s'), b, s') = 1$
4. $R'(s,a) = 0$

5. $R'(pre(s,a,s'), b) = \gamma^{-\frac{1}{2}} R(s,a,s')$
6. $\gamma' = \gamma^{\frac{1}{2}}$

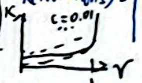
why it works: $= (\gamma')^0 \cdot R' + (\gamma')^1 R'(pre) + (\gamma')^2 U(s')$
$= \gamma^{\frac{1}{2}} \cdot \gamma^{-\frac{1}{2}} R(s,a,s') + \gamma \cdot U(s')$
$= R(s,a,s') + \gamma \cdot U(s')$

[Bellman cont] eg: R(s,a) → R(s)
- ↳ new model: $A \cup \{b\} := A'$, $S' = S \cup [pre(s,a)]_{s \in S, a \in A}$
1. $T'(s, a, pre(s,a)) = 1$
2. $T'(pre(s,a), b, s') = T(s,a,s')$



3. $R'(s) = 0$
4. $R'(pre(s,a)) = \gamma^{-\frac{1}{2}} R(s,a)$
5. $\gamma' = \gamma^{\frac{1}{2}}$

[Value iteration (mdp, ε)] (returns utility func)
1. initialise U, U', to zero | δ = max chg in utility
- repeat: U ← U'; δ ← 0
- for each state s do (in S)
- U'[s] ← R(s) … (Bellman update)
- if |U'[s] − U[s]| > δ then δ ← |U'[s] − U[s]|
- until $\delta < \varepsilon(1-\gamma)/\gamma$ ★ termination cond ← dicount factor in model
- return U
- k iterations $= \left\lceil \frac{\log(\frac{2R_{max}}{\varepsilon(1-\gamma)})}{\log(1/\gamma)} \right\rceil = \left\lceil \frac{\log(\frac{2}{(1-\gamma)})}{\log(1/\gamma)} \right\rceil$



- convergence theorem
- ↳ define max-norm $||U|| \triangleq \max_s |U(s)|$. Then, $||U - V|| = \max_s |U(s) - V(s)|$
- ↳ Let U_t := vector of utilities of all states at t-th iter:
- (Theorem). for any 2 U_t, U_t', $||U_{t+1} - U_{t+1}'|| \leq \gamma ||U_t - U_t'||$ ($||U_{t+k} - U_{t+k}'|| \leq \gamma^k ||U_t - U_t'||$)
- ↳ remarks: after bellman update, any 2 U must get closer to each other + get closer to the true U
- 2. value iteration converges to a unique, stable, opt soln when γ < 1
- (Theorem.) If $||U_{t+1} - U_t|| < \frac{\varepsilon(1-\gamma)}{\gamma}$ (local opt) → $||U_{t+1} - U|| < \varepsilon$ (global)
- ↳ let π_t be MEU policy wrt U_t
- ↳ Policy loss, $||U^{\pi_t} - U||$ is max loss by following π_t instead of π^*
- (Theorem.) $||U_t - U|| < \varepsilon \to ||U^{\pi_t} - U|| < \frac{2\varepsilon\gamma}{1-\gamma}$

[Policy iteration]
- π_0 ← arbitrary initial policy
- repeat till no Δ in π_t
- policy eval: given π_t, compute $U_t = U^{\pi_t}$
- $U_t(s) = R(s) + \gamma \sum_{s'} T(s, \pi_t(s), s') U_t(s')$ [Bellman] ∀s
- since |S| linear eqn w |S| unknowns → $O(|S|^3)$ time
- policy improvement: given U_t, find new MEU π_{t+1}
- Both iteration method can be solved manually using matrix multiplication
- PI: policy improvement can be computed by comparing utility of all actions for a given state after getting U_t (substitute in)

Transformers - form of NN architecture

- richer embedding (numerical representation of data/tokens in cont. vector space) which given vector is mapped to location depends on other vectors in the sequence (rows).
- Input: $\{x_n\}_{n=1,\dots,N}$ of D-dimensional data vectors (tokens) where $x_n = (x_{ni})^T_{i=0,\dots,D-1}$
  - → can handle mix of diff. data types (combined into joint set of tokens, no need new NN architecture)
  - → N×D data matrix X
- $\tilde{X}$ = TransformerLayer(X), $\tilde{X}$ = N×D
  - ① attention mechanism (mixes features tgt from diff token vectors across col of X (D))
  - ② acts on each row ⊥ (N) & transform features into each token vector
  - deep NN: stack multiple transformer layers w diff weights & biases to be learned using grad. descent on some loss func.

Attention weights a_{nm}

- $n=1,\dots,N$; $y_n = \sum_{m=1}^{N} a_{nm} x_m$ s.t. $\sum_{m=1}^{N} a_{nm} = 1$ & $a_{nm} \geq 0$, $n=1,\dots,N$
  - → ∀ input token x_m used to form output token y_n
  - → attention weights a_{nm} differ for diff. y_n, $n=1,\dots,N$ & depend on input data/tokens
  - → $a_{nn}=1$, $a_{nm}=0$ for $m \neq n$ → $y_n = x_n$ (input unchanged by transformation)

Breaking down components

[Dot product self-attention] → using same input seq to determine Q, K, V

- measure how much x_n (Q) attends to others x_m (keys) for $m=1,\dots,N$ by similarity/match by: $x_n^T x_m$ → ② satisfy constraints ($\sum_{m=1}^{N} a_{nm}=1$ & $a_{nm}\geq 0$)
- → soft attention: $a_{nm} = \frac{\exp(x_n^T x_m)}{\sum_{m'=1}^{N} \exp(x_n^T x_{m'})}$ normalized (one query to many K)
  - → exp is also differentiable → trainable weights
- [Matrix form] N×D output matrix $Y = \text{Softmax}(XX^T)X$
  - → comprise y_n^T in rows $n=1,\dots,N$ where softmax takes exp of every element of matrix & then norm. each row ⊥ to sum to 1
  - → X: Q, X^T: Key, X outside: V (no weights applied)

Softmax proofs: ① $\sum_{m=1}^{N} a_{nm} = \frac{\sum_{m=1}^{N} \exp(x_n^T x_m)}{\sum_{m'=1}^{N} \exp(x_n^T x_{m'})} = 1$

② since $\exp(x_n^T x_m)$ & $\sum_{m'=1}^{N} \exp(x_n^T x_{m'}) \geq 0$ also, $a_{nm} \geq 0$

property: when input vectors are orthogonal, $y_n \approx x_n$ w large $\|x_n\|_2$ (each → ∞)?

① orthogonal → $x_n^T x_m = 0$ for all $n \neq m$

② $\exp(x_n^T x_m) = 1$ $\forall n \neq m$

③ $a_{nm} = \frac{1}{N-1+\exp(x_n^T x_n)} = \frac{1}{N-1+\exp(\|x_n\|_2^2)}$ for $n \neq m$

& $a_{nn} = \frac{\exp(\|x_n\|_2^2)}{N-1+\exp(\|x_n\|_2^2)}$ for $n=1,\dots,N$

④ ∴ if $\|x_n\| \to \infty$ for $n=1,\dots,N$, then $a_{nm}=0$ for $n \neq m$, $a_{nn}=1$ for $n=1,\dots,N$

⑤ Then $y_n = \sum_{m=1}^{N} a_{nm} x_m = x_n$ for $n=1,\dots,N$

Learnable weights/params

- N×D matrix $Y = \text{softmax}(XU(XU)^T)XU$, U := D×D learning weights/params
- Limitation: $(XU)(XU)^T$ is symmetric but attention should support asymmetry
  - → apply softmax gives asymmetry, but NN is more exp w Q, K, V having ⊥ weights (instead of same U)
- ∴ $Q = XW^{(q)}$, $K = XW^{(k)}$, $V = XW^{(v)}$
  - → update: $Y = \text{softmax}(QK^T)V$
  - → limitation: softmax grad becomes exponentially small for input tokens w large magnitude or/and D → vanishing grad in backprop for deep NNs hence training more inefficient & ineffective)
  - $\left[\frac{\partial L_d}{\partial w_{jk}} = \frac{\partial L_d}{\partial o_k} \cdot \frac{\partial o_k}{\partial net_k} \frac{\partial net_k}{\partial w_{jk}}\right]$
  - as exp → ∞, softmax pushes values → 1, all others (smaller) → 0 (flat region) → grad ↓↓
  - → normalize: $Y = \text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{D_k}}\right)V$ ✓
- scale every row of matrix
  - → assume q & k vectors independent rand # w 0 mean & 1 var:

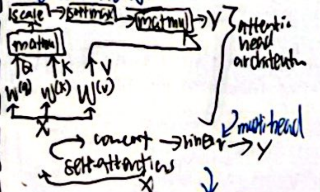
$Var(q_i k_i) = \mathbb{E}[q_i^2 k_i^2] - \mathbb{E}[q_i k_i]^2$
$= \mathbb{E}[q_i^2]\mathbb{E}[k_i^2] - 0 = 1$

$Var(QK) = \sum_{D_k} 1 = D_k$

$Var\left(\frac{QK}{\sqrt{D_k}}\right) = \frac{1}{D_k} Var(QK) = 1$

[scaled dot-prod. self-attention] NN layer → attention head

- → But mult. patterns of attention might be relevant simultaneously (eg NLP: tense & vocab. patterns)
- → Idea: Multiple Head Attention
  - → use H attention heads H_h for $h=1,\dots,H$ in parallel w ⊥ learnable weights for diff. Q, K, V matrices, then do linear transformation of their concatenation using learnable $W^{(o)}$



$H_1 H_2 \dots H_h \times W^{(o)} = Y$
N×HD$_v$ HD$_v$×D   N×D

```
For h=1,...,H do
    Q_h = X W_h^(q), K_h = X W_h^(k), V_h = X W_h^(v)
    H_h = Attention(Q_h, K_h, V_h)
H = concat(H_1,...,H_H)
return Y(X) = H W^o
```

Transformer Layer

- ↑ efficiency: residual connections + layer norm
  $Z = \text{LayerNorm}(Y(X) + X)$ (Y(X): multihead output)
  - → residual connection adds input back to output of a sub-layer
    - → "smoothing" discontinuities in loss fn grad for deep NNs → efficient & effective training (stable)
  - → layer normalization: (each of D features in $y_n + x_n$) $n=1,\dots,N$:
    $(y_{ni} + x_{ni} - \mu_i)/\sqrt{\sigma_i^2 + \delta}$, $i=0,\dots,D-1$
    - δ: small constant for numerical stability
    - $\mu_i = D^{-1}\sum_{i=0}^{D-1}(y_{ni} + x_{ni})$ post-norm
    - $\sigma_i^2 = D^{-1}\sum_{i=0}^{D-1}(y_{ni} + x_{ni} - \mu_i)^2$
    - → addresses vanishing & exploding grad
- pre-norm might yield more effective opt: $Z = Y(X') + X$, where $X' = \text{LayerNorm}(X)$
- Exp power ↑ more by post-processing output of each layer using nonlinear multilayer perceptron

$Z = \text{LayerNorm}[Y(X) + X]$
$\tilde{X} = \text{LayerNorm}[\text{MLP}[Z] + Z]$

(X → multi-head → add & norm → Z → MLP → add & norm → $\tilde{X}$)

Positional Encoding

(equivariance w.r.t input permutations)

- to handle sequential data
- add positioning encoding vector r_n (at input pos n)
  - → $\tilde{x}_n = x_n + r_n$ → unique repr. of position, boundedness, generalisation to longer seq, consistency in exp # steps between any 2 input token vectors regardless of absolute position
- Define $r_n = (r_{ni})^T_{i=0,\dots,D-1}$ where
  $r_{ni} = \begin{cases} \sin(n/L^{i/D}) & \text{if } i \text{ is even} \\ \cos(n/L^{(i-1)/D}) & \text{if } i \text{ is odd} \end{cases}$ (L>1 is user-defined)
  - → $0 < r_{ni} < 1$ (bounded)

Decoder Transformers

- each output represents a prob dist over token dict of size K thru N×K output matrix $Y = \text{softmax}(\tilde{X}W^{(p)})$ w learnable $W^{(p)}$ of D×K
  - → each softmax output unit has associated cross-entropy loss fn
- predict future tokens: each training sample is a seq of input tokens $x_1,\dots,x_n$ along w target value x_{n+1} (next token in seq)
  - → loss fn: Σ over token dict of cross entropy loss fn values summed over set of training samples, grouped into minibatches
  - → more efficient (training entire seq vs one sample at a time)
- each token: target value for seq of past tokens & input for future tokens

---

Masked Self-attention

- Masking tokens ∵ ① guide attention (wanted token) ② autoregressive, prevent future lookahead ③ mem efficiency
  - → mask by: set a_{nm} for mask tokens to −∞, so after softmax → 0
- Other pointers
  - → multiple seq stacking for parallel processing
  - → seq same length → special token <pad> to fill unused pos
  - → apply another masking (to attention weights) to ensure output tokens do not attend to any input tokens occupied by <pad> token.

Quick tips & tricks

- Proving true error/training error (COLT): helpful to have indicator var. $\delta_{h(x),c(x)} = \begin{cases} 1, & \text{if } h(x)=c(x) \\ 0 \end{cases}$
  - → Let Q(x) be distribution over X:
    $error_Q(h) = \sum_{x \in X}(1 - \delta_{h(x),c(x)})Q(x)$
    (consider dom if easier) $= \sum_{x \in X'}(1-\delta_{h(x),c(x)})Q(x) + \sum_{x \in X \setminus X'}(1-\delta_{h(x),c(x)})Q(x)$
    For X' = a set of X [may or may not be consistent]
- Proving VC(H) = d [∃ set of d pts H can shatter AND not any set of d+1 pts]:
  - ① Lower bound: VC(H) ≥ d
    - → construct a "witness" set where can achieve all 2^d labellings
  - ② Upper bound: VC(H) < d+1
    - → Take an arbitrary set d+1 pts & show any one labelling is unrealisable (eg. geometric limitation)
- semantically distinct & syntactically distinct hyp
  - → if H is H_{rd2}, "regular, depth-2 decision trees over n Boolean var (root contains same var)
    - → syntactically distinct: $2^4 \times n \times (n-1)$
    - → semantically distinct: [consider unique Bool func]
      - ① constants: all true / all false → 2
      - ② depends only on 1 var: $f(x) = x_k$ / $f(x) = \neg x_k$.
        - → n variables → 2n such functions
      - ③ depends on 2 var:
        - → $\binom{n}{2}$ unique pairs
        - → $2^4 - 2 - 4$ (1 var) distinct func output values
        - → $10\binom{n}{2} = 10\left(\frac{n(n-1)}{2}\right) = 5n(n-1)$
  - ∴ syntactically: grows at a rate of $16n^2$, semantically: grows at a rate of $5n^2$